

CloddsBot 小白指南

推文里"睡一觉醒来多 280 美元"的开源 AI 交易机器人，到底是怎么回事？——一次查证 + 拆解 + 复刻

✅ 仓库真实: alsk1992/CloddsBot

★ 查证时 2888 星 (推文时 899)

📘 TypeScript • MIT • v1.9.1

🟣 Claude 驱动 • 121 个技能包

📊 10 个预测市场 + 7 个永续交易所

⚠️ 实验仅技术层 • 不构成投资建议

📌 投资免责声明: 本指南只做"技术层"学习 (读代码、调公开只读 API、复刻检测逻辑), 全程零真实交易、零资金投入。交易机器人涉及真金白银与高杠杆, 绝大多数散户跑它亏钱而非赚钱; 推文中的"280 美元故事"无法核实且带推广链接, 请只当故事听。

0 • 先查证: 推文说的和仓库里真实有的

推文来自 X (推特) 用户 **鸟哥 @NFTCPS** (2026-09-10), 讲了一个故事: 23 岁小伙从 GitHub 找到一个 899 星的开源交易机器人, 开小号实测, 睡一觉起来账户多了 280 美元。这个故事**无法核实**, 而且推文末尾挂着 Polymarket 返佣链接 (`?r=xm1`) ——但是它指的仓库本身是真的, 而且确实有真材实料。我们逐项查证如下:

推文说的	查证结果 (GitHub API + 读源码, 2026-10-03)
"899 个 Star 的开源交易机器人"	✅ 仓库真实: alsk1992/CloddsBot , 2026-01-26 创建, 查证时已涨到 2888 星 / 339 fork (一个月翻了 3 倍多, 说明推广位很猛); TypeScript, MIT 协议, v1.9.1, 持续更新到前一天
"不预测涨跌, 专等价格和流动性异常的瞬间"	✅ 属实: <code>src/arbitrage/</code> 737 行跨平台套利服务, 只比价不下判断; README 自述 "Scans for edge, executes instantly"
"监控 Polymarket + 监控 Binance"	✅ 属实: 接了 10 个预测市场 (Polymarket、Kalshi、Manifold、Metaculus、PredictIt、Betfair、Drift 等) + 7 个永续交易所 (Binance、Bybit、Hyperliquid、MEXC 等)
"风险太高直接取消不玩"	✅ 属实且比推文猛得多: <code>docs/RISK_MANAGEMENT.md</code> 里有 10 道盘前检查的风控引擎 (熔断、VaR、波动率分档、Kelly 仓位.....见疑问 9)
"早上打开 Telegram, 满屏成交订单"	✅ 机制属实: 接了 21 个聊天平台 (Telegram/Discord/微信类...), 成交推送是真的功能
"睡一觉多 280 美元"	❌ 无法核实, 纯属故事; 且推文带交易所返佣链接, README 顶部还挂着一个 meme 币合约地址 ("Clodds CA") ——典型的 "开源项目 + 币 + 返佣"三件套推广套路

一句话定性: 仓库是真开源项目、代码有干货, 值得我们拆开学技术; 但推文是带推广性质的"成功学包装", 赚不赚钱与开源与否毫无关系。下面所有实验都只动"只读"的公开行情接口, 一分钱不投。

1 • 工科毕业生的 14 个疑问 (本指南目录)

#	疑问	一句话答案
Q1	CloddsBot 到底是个啥？	能用聊天对话驱动的 AI 交易终端
Q2	和已学的 NautilusTrader / stock 工具箱什么关系？	量化世界的第三种形态：AI Agent 终端
Q3	"不预测涨跌也能赚"是什么原理？预测市场是啥？	预测市场 = 用真金白银投票的未来概率机
Q4	为什么 YES 卖价 + NO 卖价 < 1 就稳赚？	负风险套利 (negative risk)
Q5	机器人怎么知道"两个市场说的是同一件事"？	语义匹配 (Jaccard 相似度)
Q6	为什么实测扫不到套利机会？	做市机器人把地板焊死在 1.001 (市场有效)
Q7	跨交易所搬砖为什么不能闭眼搬？	净边际 = 毛价差 - 手续费, 实测为负
Q8	三角套利怎么用算法找？	-log 变边权 + 找负环 (Bellman-Ford)
Q9	"风险太高直接不玩"具体是怎么实现的？	RiskEngine 十道盘前检查 + Kelly 仓位
Q10	一条聊天消息怎么变成一笔真实订单？	九步交易数据流
Q11	121 个"技能包"是什么？和 Claude Code 技能什么关系？	同款 SKILL.md 格式, 四层优先级加载
Q12	x402 是什么新词？	机器对机器付款的"代理商务"协议
Q13	这种"开源 + 币 + 返佣"的项目怎么识别套路？	四个红旗信号 + 查证三步法
Q14	我能拿它做什么？千万别做什么？	读代码学架构 ✓ · 真金白银跑它 ✕

Q1 CloddsBot 到底是个啥？

Claude + Odds (赔率) = Clodds。一句话：**一个跑在自己电脑上的 AI 交易终端——你用聊天口吻下指令，它替你盯市场、找机会、下单、报风险**。它不是一个"策略"，而是一个把「行情数据 + 交易所接口 + AI 大脑 + 风控引擎 + 聊天界面」打包在一起的操作台。

没有它之前：想在 Polymarket 和币安同时搬砖的人，得自己写爬虫、自己接 WebSocket、自己写风控、自己搭 Telegram 机器人——四套活儿。CloddsBot 把这四套活儿全做成了现成模块，还加了个能听懂人话的 AI 大脑 (Claude) 当总调度。

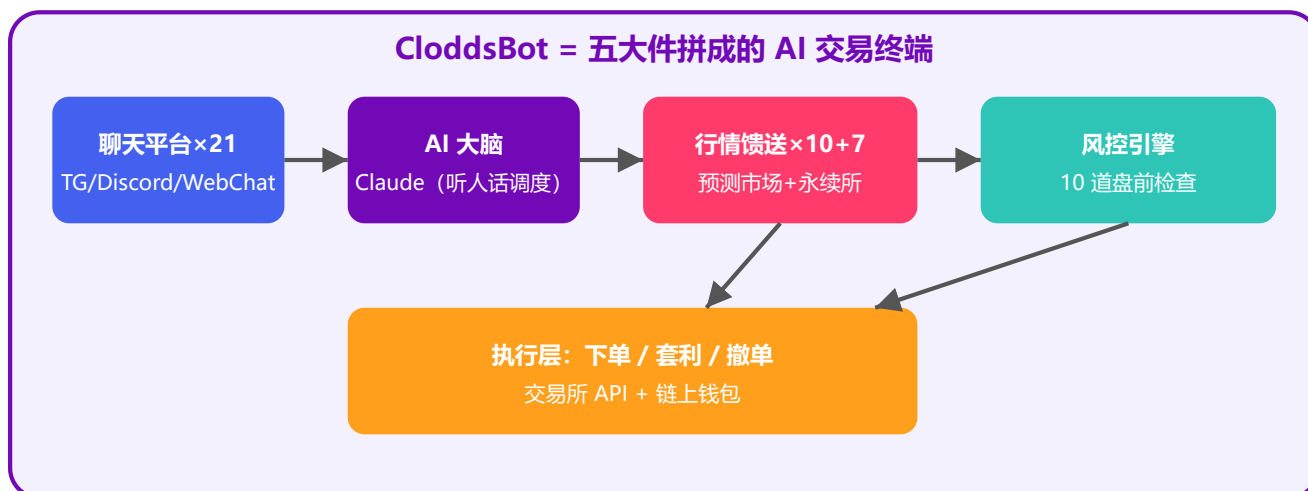


图1: CloddsBot 五大件——聊天入口、AI 大脑、行情馈送、风控引擎、执行层

记住这句：它卖的不是"赚钱策略"，是"基建"。策略好不好另说，**把四套基建活儿打包开源**才是它 2888 星的真正原因。

Q2

和已学的 NautilusTrader、stock 工具箱是什么关系？

我们已经学过量化世界的两种形态（见知识库 [[事件驱动回测]] 和 [[个人量化工具箱范式]]），CloddsBot 是**第三种**：

	机构框架派 NautilusTrader	个人工具箱派 Rockyzsu/stock	AI Agent 终端派 CloddsBot（本篇）
定位	正规军：回测/实盘同码的事件驱动引擎	游击队：采集→落库→分析→下单脚本集	顾问型 ：AI 大脑听人话、调工具、管风控
策略从哪来	你写代码实现 Strategy 类	你抄作者十年实盘脚本	AI 读市场数据现判 + 118 个内置策略模块
交互方式	Python API / 配置	改脚本跑脚本	自然语言聊天 ："买 100 刀 Trump 赢"
市场	外汇/加密/A股全 AssetClass	A股/可转债/基金	预测市场 + 加密永续 + Solana 链上

三者共享同一批底层命题：**数据从哪来**（对比 [[数据源降级链]]）、**成本怎么算**（[[交易成本与鞭打效应]]）、**风险怎么拦**（[[护栏模式Guardrails]]）、**AI 管什么不管什么**（[[事实与判断分离]]——CloddsBot 的风控全部是确定性代码，AI 只在"选哪个机会"层面说话，这和 daily-stock-analysis 的做法同源）。

类比记忆：NautilusTrader 是**赛车引擎厂**，stock 工具箱是**老师傅的修车工具墙**，CloddsBot 是**会聊天的自动驾驶套件**——车还是那台车（交易所），但方向盘换成了 AI。

Q3

"不预测涨跌也能赚"是什么原理？预测市场是啥？

推文最反直觉的一句是"它根本不预测涨跌"。要理解这个，先懂**预测市场 (prediction market)**：把一个未来事件（"BTC 一小时内会涨吗？”）变成一份**二元合约**——

- 市场发行两种股份：**YES 股**和 **NO 股**，价格在 0~1 美元之间浮动；
- 事件揭晓后，赢的那股固定赔 **1.00 美元**，输的那股归零；
- 所以 **YES 的价格 = 市场公认的该事件发生概率**（YES 卖 0.65 刀 $\approx$ 大家认为 65% 会发生）。

Polymarket 是目前最大的加密预测市场，日成交量上亿刀。它有个天然特性：**短期 BTC 涨跌盘**（5 分钟 / 15 分钟 / 1 小时一轮）全天候滚动开盘——这就是机器人"晚上也不用睡"的原因：市场永远开着。

预测市场：把"未来事件"变成可交易的股份**YES 股 现价 \$0.65**

事件发生 $\rightarrow$ 赔 \$1.00（赚 0.35）
没发生 $\rightarrow$ 归零

NO 股 现价 \$0.36

事件没发生 $\rightarrow$ 赔 \$1.00（赚 0.64）
发生了 $\rightarrow$ 归零

YES 价格 + NO 价格 $\approx$ 1.00（定价的引力锚点）—— 偏离它，机会就来了

图2：二元合约的两股定价。两边价格之和被"必赔 1 美元"锚在 1.00 附近

关键转变：机器人不回答"会涨吗"，它只回答"**两边的报价加起来是不是错位了**"——判断题外包给了全市场的交易者，机器人只做**算术题**。这就是"不预测也能赚"的全部秘密（前提是机会真存在，见 Q6）。

Q4 为什么 YES 卖价 + NO 卖价 < 1 就稳赚？

这就是推文里机器人的核心打法，行话叫**负风险套利 (negative risk arbitrage)**。逻辑是一道小学算术：

- 同时买 1 股 YES (\$0.48) + 1 股 NO (\$0.49)，总成本 **\$0.97**；
- 事件揭晓必然二选一：赢的那股**固定赔 \$1.00**；
- 所以无论结果如何，你收 \$1.00，**稳赚 \$0.03，约 3.1% 无风险收益**。

同平台内出现这种错位叫负风险套利；同一个事件在 Polymarket 卖 YES、在 Kalshi 买 NO 出现错位，叫**跨平台套利**——CloddsBot 的 `src/arbitrage/` 两种都扫，这正是“监控 Polymarket”那一块的真身。

```
repo 源码对应 (src/arbitrage/index.ts)
ArbitrageOpportunity {
  buyPrice, sellPrice,      // 两腿价格
  spread, spreadPct,        // 价差与百分比
  profitPer100,             // 每 100 刀本金的理论利润
  confidence,               // 匹配置信度(0~1)
  isActive                  // 机会是否仍有效(转瞬即逝)
}
```

但注意：理论稳赚 ≠ 真赚。三只手会伸过来：①手续费 ②下单滑点 ③资金占用期的事件风险（平台跑路/规则改）。所以工程上必须配“净边际过滤”（Q7）和风控（Q9）。**先记住公式：净赚 = (1 - YES卖价 - NO卖价) - 手续费 - 滑点**，下一节的实验就靠它。

Q5 机器人怎么知道"两个市场说的是同一件事"?

跨平台套利有个脏活：Polymarket 上写 "Will the Fed cut rates in October?", Kalshi 上写 "Fed rate decision October meeting", 机器人得认出**这是同一件事**才能对冲。CloddsBot 的做法在 `questionSimilarity()` 函数里, 意外地朴素:

```
// src/arbitrage/index.ts (真实源码, 简化注释)
function questionSimilarity(q1, q2) {
  const normalize = s => s.toLowerCase()           // ① 全小写
    .replace(/[,.']/g, ' ')                       // ② 去标点
  const words1 = new Set(normalize(q1))           // ③ 分词成集合
  const words2 = new Set(normalize(q2))
  const inter = [...words1].filter(x => words2.has(x)) // ④ 交集
  return inter.size / union.size                   // ⑤ Jaccard 相似度
}
```

Jaccard 相似度 = 交集 ÷ 并集。上面两句分词后交集有 {fed, rate, october}, 并集 9 词, 相似度 0.33, 超过阈值就列为"候选同事事件", 再由 AI 大脑 (Claude) 做最终语义确认。**确定性算法海选 + LLM 终审**——和我们在 [[LLM裁判与自动评估]]、[[事实与判断分离]] 里学过的分工完全一致: 便宜精确的先干粗活, 贵而聪明的只干细活。

两平台"同事事件"识别: Jaccard 海选 + AI 终审

Polymarket: "Will the Fed cut..."

Kalshi: "Fed rate decision..."

Jaccard = 交集/并集 > 阈值? → 候选对
→ Claude 语义终审 → 建立配对, 开始比价

图3: 市场配对流水线——纯文本相似度做粗筛, 大模型做终审

工程亮点: 粗筛必须便宜 (每轮要扫上千个市场), 终审才准花钱。把"贵而聪明"的调用次数压到 1% 以下, 是所有 AI 应用省钱的同一招。

Q6 为什么实测扫不到套利机会？——地板被焊死在 1.001

我们照着 repo 的套利逻辑写了 Python 复刻版去扫 Polymarket 真实行情（实验 ex1，见实验节），结果 25 个最热门的市场里**一个机会都没有**——所有市场的 YES 卖价 + NO 卖价 全部落在 **1.001 ~ 1.010**。为什么？

- Polymarket 价格最小变动单位（**tick**）是 **\$0.001**；
- 成百上千个做市机器人 7×24 挂单，一旦双边合计跌破 1.001，**零点几秒内就被别人的机器人吃掉**；
- 所以留给我们的“地板”恰好是 $1 \times \text{tick} = 1.001$ ——这个数字就是市场有效性的体温计。

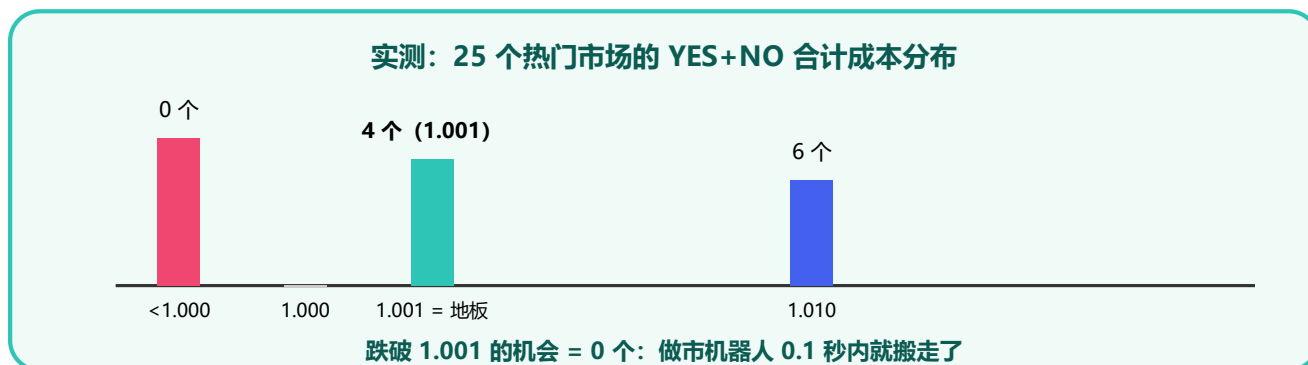


图4：ex1 实测分布（12/25 市场成功取到双边报价，全部 ≥ 1.001 ）

这一课最值钱：推文里“睡一觉赚 280 刀”如果真发生过，赚的不是“捡钱”，是**速度、门槛（tick）和费率之间的极限缝隙**——拼的是服务器离交易所近、下单快。散户拿家用电脑 + 手动操作，处在食物链最底端。**看见“无风险稳赚”四个字，先问 tick 和手续费。**

Q7 跨交易所搬砖为什么不能闭眼搬？——净边际

第二块"监控 Binance"的打法是**跨所搬砖**：同一资产（如 BTC/USDT）在 A 所便宜 B 所贵，A 买 B 卖。但 repo 的 V2 套利文档（docs/V2_HFT_ARBITRAGE.md）第一原则就是**净边际过滤（net-edge filter）**：

净边际 = 毛价差 - 买入所手续费 - 卖出所手续费 - 滑点 - (资金到账延迟 × 波动率)

repo 还额外惩罚：报价过期(stale-quote)、库存不足、延迟高的路径 — 全部不过门槛直接拒绝出单

我们用 OKX 和 Kraken 的真实行情做了实测（实验 ex2）：两个大所的 BTC 毛价差只有 **0.0007%~0.011%**，而两腿 taker 手续费合计 **0.36%**——**净边际 -0.36%，两个字：不干**。这就是为什么 repo 里写"Rejects plans that fail minimum net-edge"——大多数"看起来有价差"的机会，死在手续费这道门。

ex2 实测瀑布：毛价差是怎么被一层层吃掉的

毛价差 0.0007% ~ 0.011% (小到几乎看不见)

- 双腿 taker 手续费 0.36% (OKX 0.1% + Kraken 0.26%)

= 净边际 -0.36% → RiskEngine 拒绝执行 X

图5：净边际瀑布——绿色（毛利）还没橙色（成本）长，必亏

联动已学：这正是 [[交易成本与鞭打效应]] 的又一实证——信号越密、交易越频繁，成本越是生死线。**任何"搬砖"策略上市前，第一笔要算的账永远是手续费。**

Q8 多跳套利怎么用算法找？——负环检测

repo 的 V2 还有个更炫的模块：**多跳套利 (multi-hop arbitrage)**。不只比两个价格，而是在“货币兑换图”里找**环路**：BTC → USDT → ETH → BTC，绕一圈回来如果钱变多了，就是三角套利。经典算法两步：

- **第一步：汇率取负对数变边权。**把每一跳的汇率 r 乘上 $(1 - \text{手续费})$ ，取 $w = -\log(r \times (1 - \text{fee}))$ 。这样“绕一圈的汇率乘积 > 1 ”就等价于“绕一圈的边权之和 < 0 ”——对数把乘法变加法；
- **第二步：找负环。**在带负权的图里找“边权和为负的回路”，标准算法是 **Bellman-Ford**（它能处理负权边，第 n 轮还在松弛就说明有负环）。

我们用 OKX 真实行情三元组（BTC/USDT、ETH/USDT、ETH/BTC）实测（实验 ex3）：两个方向的环扣掉每跳 0.1% 手续费后，边权和 **+0.0030**，资金乘数 **0.997**——绕一圈亏 0.3%，无套利。主流大所的三角缝隙同样被机器人抹平。

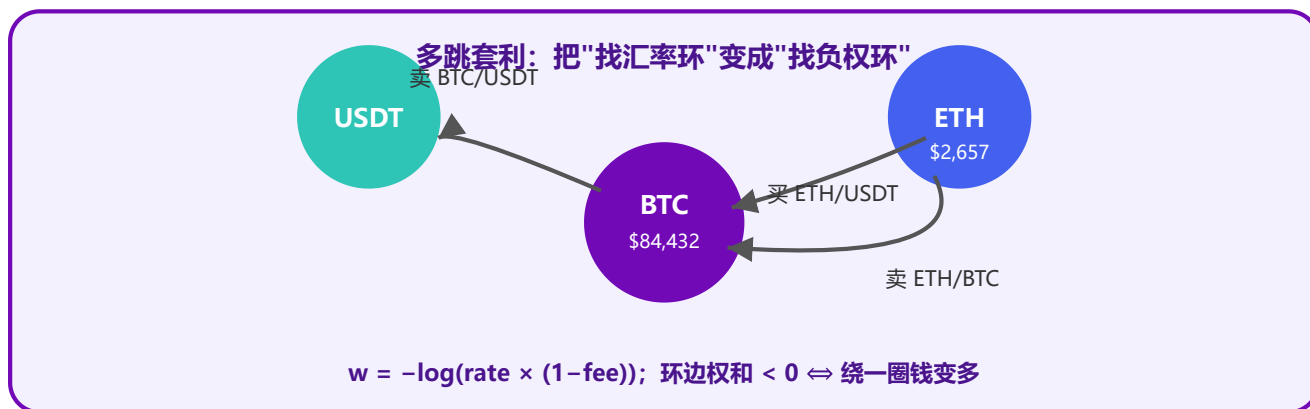


图6：ex3 实测的三元组环——两个方向边权和均为正，无套利

为什么这设计漂亮：把金融问题（哪个环赚钱）转成图论问题（有没有负环），就能用几十年打磨过的成熟算法一次扫全图。repo 还在此之上做**延迟感知排序**——同样无套利时优先选延迟低的路径，因为套利的本质是抢时间。

Q9

"风险太高直接不玩"具体怎么实现？——十道盘前检查

推文第三块"实时盯流动性，风险太高直接取消"，对应 repo 里最工程化的部分：`src/risk/engine.ts` 的 **RiskEngine**。每一笔单子下单前必须按顺序过 10 道检查，任一道拦截即否决：

#	检查	干嘛的（白话版）
1	Kill switch 总闸	一键停止交易开关有没有被拉下
2	熔断器 circuit breaker	连续亏损/异常行情时自动断电
3	单笔上限	这一单金额超没超上限
4	敞口限额	总持仓超没超上限
5	日亏/回撤/集中度	今天亏够数没有、是不是全押一个宝
6	VaR 风险价值	"95% 的情况下今天最多亏多少"超没超线
7	波动率分档	低/常/高/极端四档，极端档减仓到 25% 或直接停机
8	Kelly 仓位	按胜率赔率算"该押多大"，不批准只调大小

Kelly 公式一句话：**押注比例 = 优势 ÷ 赔率**。胜率 55%、赔率 1:1 时 Kelly 仓位 = $(0.55 - 0.45) / 1 = 10\%$ 本金；实际工程用"半 Kelly"（再砍半）因为胜率估计本身有误差。repo 里 `src/trading/kelly.ts` 做的是**动态 Kelly**——用最近的滚动胜率实时调整，连击亏损时自动缩仓。

这个设计和我们已学的 [[护栏模式Guardrails]]、[[评分契约与护栏]] 一脉相承：**AI 负责"挑哪个机会"，确定性代码负责"这笔单能不能出"——权力边界用架构焊死，不靠 AI 自觉**。仓库的 `tests/trading` 里甚至有专门的 "shared pre-trade gate" 测试：风控减仓单可以超仓位上限，但 kill switch 谁也绕不过（我们实测 11/11 全过，见实验 ex4）。

记住分层：预测（AI，可以错）→ 定量（Kelly，公式算）→ 放行（10 道检查，一票否决）。**三层里任何一层交给 AI 全权负责，都是事故。**

Q10

一条聊天消息怎么变成一笔真实订单？——九步数据流

在 Telegram 里输入 "Buy \$100 of YES on Trump winning", 到订单成交, repo 的 docs/ARCHITECTURE.md 定义了完整九步:

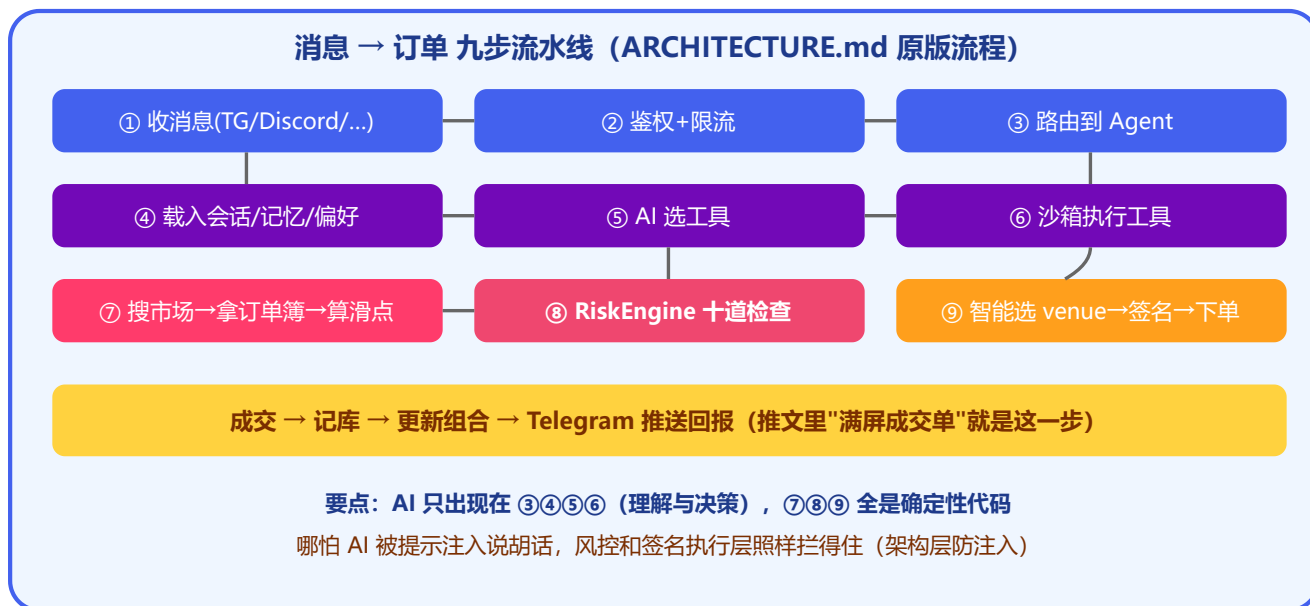


图7: 九步数据流——AI 与确定性代码的分界线画在第 ⑥⑦ 步之间

与已学联动: ①~⑥ 就是 [[Agent循环]] + [[LLM工具调用]] 的标准套路; ⑧ 的"一票否决"是 [[护栏模式 Guardrails]]; "带着钱包的 Agent 必须架构层防 [[提示注入]]"是本仓库给全行业的示范。

Q11

121 个"技能包"是什么? 和 Claude Code 技能什么关系?

CloddsBot 的功能扩展机制和我们已经在用的 **SKILL.md 技能包** (见 [[AgentSkills技能包]]) 是**同一套格式**: 一个文件夹 + 一个带 YAML frontmatter 的 markdown 说明文件, 加载器自动发现、无需注册。README 自称 "Full backwards compatibility with OpenClaw-format SKILL.md files"——整个生态在合流。

src/skills/bundled/arbitrage/SKILL.md ← 121 个内置技能之一

```

---
name: arbitrage
description: "跨市场套利扫描"
---
# Arbitrage
| 命令 | 说明 |
| /arbitrage scan | 扫描当前套利机会 |
  
```

四层优先级加载 (同名高优先级覆盖): **工作区 skills/ > 用户 .clodds/skills/ > 配置 extraDirs > 内置 bundled/**——这正是 [[分层按需加载]] + [[能力层与后端路由]] 的思路: 入口常驻很小, 大能力按需唤醒, 换后端不改入口。

视线拉高: 一个交易终端, 用的却是通用 Agent 生态的技能格式。这意味着"给 Agent 装新本事"这件事已经出现**跨行业的统一接口**——今天学的 SKILL.md 写法, 在交易、办公、机器人领域通用。

Q12 x402 是什么新词?

仓库 topics 里有个陌生的词 **x402**: 一个"机器对机器支付"的开放协议, resurrect 了 HTTP 402 (Payment Required) 这个几十年没用的状态码——API 被调用时可以直接回一句"付费 0.001 美元再试", 机器 A 用加密钱包当场付钱、当场拿到服务, **不需要人刷卡、不需要注册账号**。业内叫它"代理商务 (agentic commerce)": 当 AI Agent 们开始互相买服务, 就得有机器间的收银台。

CloddsBot 自称 "Agent commerce protocol for machine-to-machine payments", 设想是: 你的机器人缺一条付费数据源, 它自己发现、自己砍价、自己付款。这类协议还在早期, 但方向值得知道: **AI 的下一个基建不是更强的模型, 而是"付钱的手"**。

联想记忆: [[MCP协议]] 解决"Agent 怎么调工具", x402 想解决"Agent 怎么为工具付费"。一个管握手, 一个管收银。

Q13 "开源 + 币 + 返佣"的推广套路怎么识别?

这次查证本身就是一个现成的"鉴伪流程", 四红旗 + 三步法:

红旗信号 (本项目全中)	查证三步法 (本次实际操作)
① 推文带交易所 返佣链接 (?r=xml) ② README 顶部挂 meme 币合约地址 ("Clodds CA") ③ 星数短期暴涨 (月内 899→2888) ④ 讲"普通人逆袭"故事但 零可核实证据 ("23 岁小伙赚 280 刀"查无此人)	① fxtwitter 拿推文全文 , 把链接和关键数字全部列出来 ② GitHub API 核仓库 : 创建时间、语言、协议、commit 频率、星数曲线——判断"代码是不是真的有人在维护" ③ 读源码 + 跑测试 : 文档吹得再响, typecheck 过不过、测试绿不绿、核心逻辑在哪个文件多少行, 一跑便知

结论要分两层说: 仓库本身——真的、活跃的、代码质量中上的工程 (这点可信); 仓库的**盈利叙事**——营销话术, 不可信。 **"代码是真的"和"故事是真的"是两个独立命题, 必须分开验证。**

Q14 我能拿它做什么？千万别做什么？

可以做的（零风险高收益）：

- 把它当**AI Agent 终端的参考架构**读：网关/渠道适配/技能包/风控引擎的分层，搬到任何 Agent 项目都成立；
- 跑它的**单元测试**学风控测试怎么写（kill switch 优先于一切）；
- 用它的**公开 API 思路**（gamma + CLOB 都免钥只读）做行情分析、数据可视化、市场有效性研究；
- 复刻套利检测器当**教学工具**——本指南实验就是范例，拿来改阈值、加市场、画图都行。

千万别做的（血泪忠告）：

- ❌ **充真金白银跑实盘**：200x 杠杆、链上钱包私钥交给一个 GitHub 项目，等于把银行卡密码贴朋友圈；
- ❌ 买它 README 里的 **meme 币**（那是项目方的融资手段，不是投资机会）；
- ❌ 相信“挂机睡觉赚钱”——所有 **passive income 叙事在套利领域都意味着你在食物链底端**（Q6/Q7/Q8 三连实测已证明：热门市场无缝隙，缝隙拼速度和费率）；
- ❌ 在没有 10 道风控检查的情况下裸跑任何交易脚本。

一句话总结：这个仓库最值钱的产出不是利润，是一套可以直接抄的“AI + 交易”工程蓝图——蓝图免费，学费免交。

2 · 这个项目给我什么（能力清单）

① 一张 AI 交易终端架构图

网关→渠道适配→Agent→工具→风控→执行六层，可直接套用到任何“AI + 危险操作”的系统设计。

② 负风险套利的完整算法思路

YES+NO<1 的检测公式、Jaccard 市场配对、置信度打分——300 行 Python 就能复刻（本指南 ex1 就是证据）。

③ 净边际过滤公式

毛价差-手续费-滑点-延迟惩罚，任何交易/搬砖策略上市前的第一笔账。

④ 负环检测找多跳套利

-log 边权 + Bellman-Ford 的图论转化套路，可推广到任何“环路收益”类问题。

⑤ 十道盘前风控清单

Kill switch→熔断→限额→VaR→波动率分档→Kelly，风控测试的黄金模板（tests/trading 可直接读）。

⑥ “开源+币+返佣”鉴伪流程

fxtwitter 取证 → GitHub API 验仓 → 跑测试代码，三步分离“真代码”与“假故事”。

3 · 动手实验（全部真实运行，零交易零API费）

实验环境：Windows 11 + Python 3.14.7（纯标准库，零第三方依赖）+ Node v24.21.0。所有脚本在 cloddsbot\exercise\。数据全部来自公开只读接口，全程无任何真实交易，不构成投资建议。

实验 ex1 • 复刻"监控 Polymarket": 负风险套利扫描器 (主实验)

第 1 步 • 理解要复刻什么

目的: 把 repo `src/arbitrage/index.ts` 的五步流水线 (扫描→比价→算边际→打分→风控过滤) 用 100 行 Python 复刻。

操作: 读 repo 源码的 `ArbitrageOpportunity` 接口与 `calculateArbitrage()`, 然后打开 `exercise\ex1_poly_arb.py`。

实现了什么: Polymarket gamma-api 拉热门市场 + CLOB 公开订单簿取双边最优卖价。

验证: 脚本第一行输出"拉到 40 个活跃市场"。

第 2 步 • 运行扫描

操作: `python ex1_poly_arb.py`

目的: 用真实行情算每个市场的 YES 卖价 + NO 卖价。

实现了什么: 12 个市场成功取到双边报价 (其余因订单簿空的被跳过)。

验证 (真实输出摘要):

```
[1] 市场扫描: 拉到 40 个活跃市场, 其中二元市场 40 个
[2] 价格比较: 成功拿到 12 个市场的 YES/NO 双边最优卖价
[3] 风控过滤: 流动性>=5000$ 且 净边际>=0.005 且 边际<=0.15 -> 通过 0/12
Will Flávio Bolsonaro win... YES 0.570 NO 0.431 合计 1.001 毛边际 -0.001
Will the Fed increase...      YES 0.005 NO 0.996 合计 1.001 毛边际 -0.001
2026 Balance of Power...      YES 0.080 NO 0.930 合计 1.010 毛边际 -0.010
```

第 3 步 • 改参数再跑 (可见成果)

操作: 把 `MIN_EDGE` 从 0.005 改成 -0.02 (故意放进负边际"机会"), 重跑。

实现了什么: 表格立刻出现一堆"合计 1.01"的假机会被排到前面——直观看到"没有风控过滤会怎样"。

验证: 过滤行从"通过 0/12"变成"通过 12/12", 证明过滤逻辑在起作用。

第 4 步 • 解读

所有市场合计成本 ≥ 1.001 = tick 地板 (Q6 结论)。✅ 预期结果: 热门市场零套利机会——市场有效, "捡钱"不存在。

实测坑: gamma API 的 `clobTokenIds` 字段返回的是 **JSON 字符串** 不是数组, 直接 `len()` 永远等于 1, 必须再 `json.loads()` 一次——我们第一版就栽在这, 40 个市场全被过滤成 0 个。

实验 ex2 • 复刻"监控 Binance": 跨所净边际计算

操作与目的

`python ex2_venue_edge.py` —— 用 OKX 和 Kraken 公共行情算 BTC/USDT 跨所搬砖的净边际 (Binance 公共 API 实测被地域限制 "restricted location", 本身就是个真实世界坑: 交易所会按 IP 拒服务)。

验证 (真实输出) :

```
OKX      ask=84,432.6  bid=84,432.5
Kraken   ask=84,433.2  bid=84,423.2
A: Kraken买 -> OKX卖   毛边际 -0.0008%  双taker费 0.3600%  净边际 -0.3608%  拒绝
B: OKX买 -> Kraken卖   毛边际 -0.0111%  双taker费 0.3600%  净边际 -0.3711%  拒绝
=> 印证 repo 净边际过滤存在的意义: 大多数'看起来有价差'的机会过不了手续费这道门
```

✅ 预期结果: 两大所毛价差 0.001% 量级 << 0.36% 双腿手续费, 两个方向都被拒绝——"搬砖梦"被手续费当场击碎。

实验 ex3 • 多跳套利: 负环检测

操作与目的

`python ex3_multihop.py` —— 拿 OKX 真实三元组 (BTC/USDT、ETH/USDT、ETH/BTC) 做 $-\log$ 边权 + Bellman-Ford 思想的负环检测。

验证 (真实输出) :

```
BTC-USDT mid = 84,432.55  ETH-USDT mid = 2,657.59  ETH-BTC mid = 0.0315
环1 BTC->USDT->ETH->BTC  边权和=+0.003028  资金乘数=0.996977  无套利
环2 反向 BTC->ETH->USDT->BTC 边权和=+0.002975  资金乘数=0.997029  无套利
=> 三角套利在主流大所早被机器人抹平
```

✅ 预期结果: 扣每跳 0.1% 手续费后两个方向边权和均为正 (绕一圈亏 0.3%) ——图论算法正确工作, 市场同样有效。

实测坑: 第一版把环方向汇率写反 (BTC→USDT 用了 1/价格), 负环"满地都是"资金乘数 71 亿倍——**套利检测器最危险的 bug 就是方向搞反后把必亏报成暴富**。正确语义: $\text{rate} = \text{换到的资产数} \div \text{换出的资产数}$ 。

实验 ex4 · 跑通上游仓库：typecheck + 单元测试

操作与目的

克隆 `git clone https://github.com/alskl992/CloddsBot` repo (v1.9.1, commit c930628) → `npm install` → `npm run typecheck` → 跑交易风控相关测试。

验证 (真实输出) :

```
npm install → 834 个包 (坑: helius-laserstream@0.1.8 只支持 darwin/linux,
  Windows 需 npm install --force 绕过平台检查)
npm run typecheck → 0 errors
npx tsx --test tests/trading/*.test.ts
  ✓ allows risk-reducing exits over the size cap but still honors kill switches
  ✓ shared pre-trade gate (2.5ms)
  ✓ Solana swap notional valuation (2.1ms)
tests 11 / pass 11 / fail 0
```

✅ 预期结果：类型检查全过 + 风控测试 11/11 全绿——开源代码的“真实性”最终由测试证明，不是由 README 证明。测试里最有含金量的一条：“**风控减仓单可以超仓位上限，但 kill switch 任何情况下都拦得住**”——这就是 Q9 十道检查的活标本。

4 · 知识入库指引

本篇沉淀 5 个新概念进 vault：**预测市场与二元合约**、**负风险套利**、**净边际与多跳套利环**、**风控链与 Kelly 仓位**、**AI 交易终端**；与已有概念互链：[[事件驱动回测]]、[[交易成本与鞭打效应]]、[[个人量化工具箱范式]]、[[护栏模式 Guardrails]]、[[AgentSkills 技能包]]、[[LLM 工具调用]]、[[提示注入]]。

CloddsBot 小白指南 · 2026-10-03 生成 · 拾遗学习流水线 task 6ace9065 · 数据与输出均为 2026-10-03 真实实测 · 仅供学习，不构成投资建议